HAI913I · Évolution et restructuration des logiciels

TP 1 · Annexe 3 — Le patron Visitor, ASTVisitor et Spoon

Année 2026–2027 Enseignant Bachar Rima

À QUOI SERT CETTE ANNEXE

Comprendre *pourquoi* JDT fait écrire des classes qui héritent d'`ASTVisitor`, ce que change la valeur renvoyée par `visit`, et *comment* organiser ses visiteurs dans l'analyseur du TP. L'annexe part du patron Visitor du GoF sur un petit arbre d'expressions (structure, code complet, exécution), montre le même mécanisme dans JDT, compare les façons d'utiliser un visiteur dans la chaîne d'analyse, puis situe brièvement Spoon. Tous les exemples sont dans `resources/visitor-examples` et s'exécutent avec les commandes de son `README.md` (JDK 17 ou 21).

1. Le problème

Un analyseur statique applique **beaucoup d'opérations différentes** (compter des méthodes, lister des appels, mesurer des lignes…) à une **structure d'objets stable** : l'arbre syntaxique d'un programme Java, dont les types de nœuds ne changent qu'avec le langage. Deux solutions naïves :

- ajouter une méthode par opération dans chaque classe de nœud : l'API de l'arbre grossit à chaque nouvelle analyse, et chaque analyse est éparpillée dans des dizaines de classes ;
- une cascade de `instanceof` dans chaque analyse : on oublie un cas sans que le compilateur le signale.

Le patron **Visitor** (Gamma, Helm, Johnson et Vlissides, 1994) regroupe chaque opération dans une classe à part, sans modifier les classes de nœuds.

2. Le patron Visitor

2.1. Structure générique

Le patron sépare deux hiérarchies : celle des **éléments** de la structure (qui changent peu) et celle des **visiteurs** (une classe par opération).

RÔLE (GOF)	RESPONSABILITÉ	DANS L'EXEMPLE DE LA § 2.2
Element	déclare `accept(visiteur)`	interface `Expr`
ConcreteElement	implémente `accept` en appelant la visite qui porte son type	`Num`, `Add`, `Mul`
Visitor	déclare une méthode de visite par élément concret	`ExprVisitor<R>`
ConcreteVisitor	implémente **une opération** complète sur toute la structure	`Evaluator`, `Printer`
ObjectStructure	l'ensemble parcouru	l'arbre de `2 * (3 + 4)`
Client	construit la structure, choisit le visiteur, lance `accept`	`GofDemo`

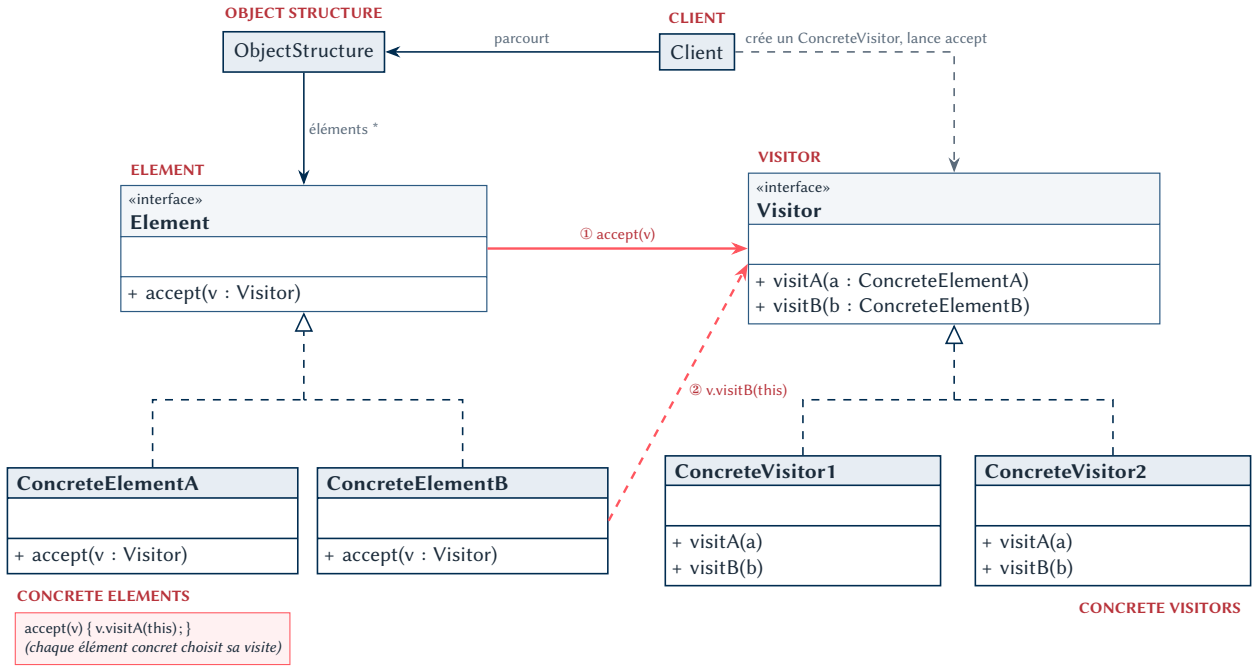


Figure 1 – Structure du patron Visitor. Les deux flèches corail numérotées forment le *double aiguillage* : ① l'élément reçoit un visiteur, ② l'élément concret rappelle la méthode du visiteur qui porte son propre type.

2.2. Sur un arbre d'expressions

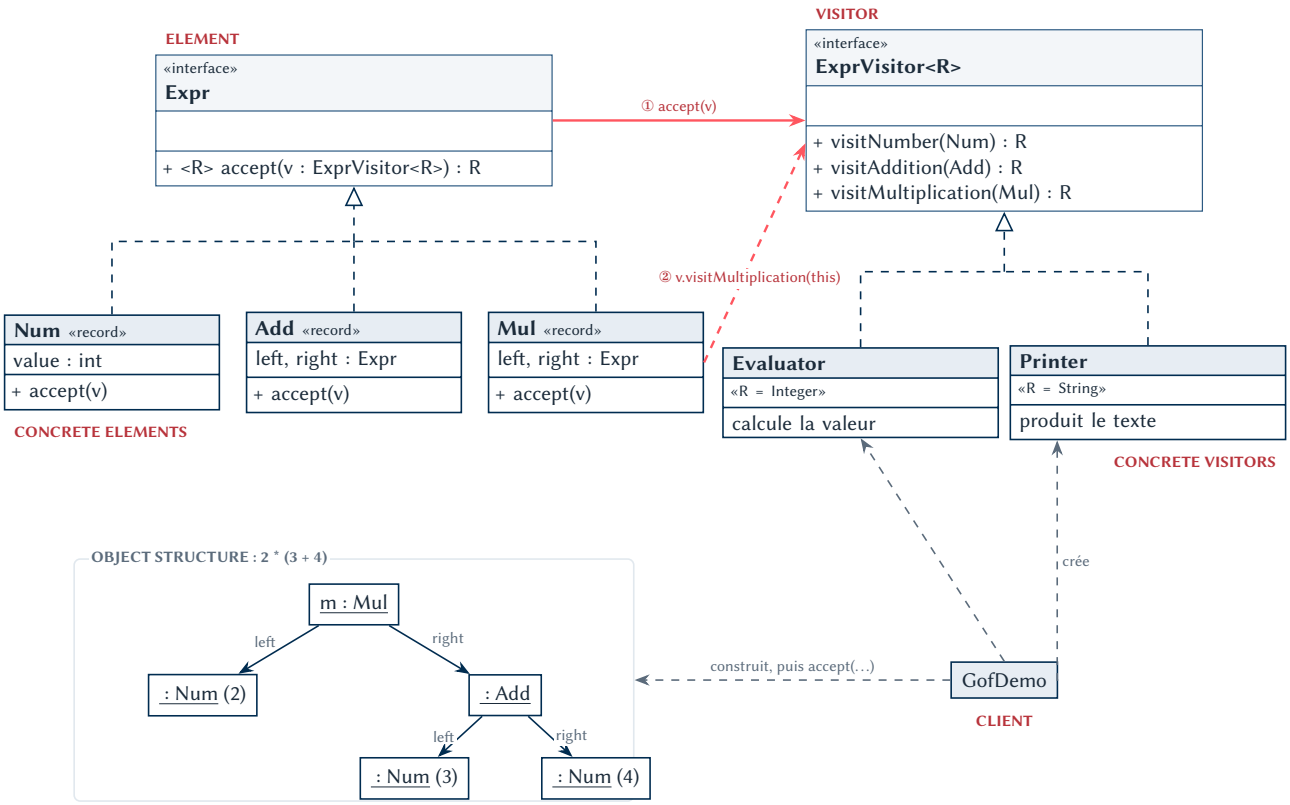


Figure 2 – Les rôles du patron appliqués à l'exemple exécutable. En bas, la structure d'objets réellement construite par `GofDemo` pour `2 * (3 + 4)`.

2.3. L'exemple complet

Le projet contient tous les rôles. L'élément et le visiteur :

Listing 1 – gof/Expr.java — Element

```java
public interface Expr {

    <R> R accept(ExprVisitor<R> visitor);
}
```

Listing 2 – gof/ExprVisitor.java — Visitor

```java
public interface ExprVisitor<R> {

    R visitNumber(Num number);

    R visitAddition(Add addition);

    R visitMultiplication(Mul multiplication);
}
```

Les trois éléments concrets. Chacun ne fait qu'une chose : appeler la visite qui porte *son* type.

Listing 3 – gof/Num.java — ConcreteElement

```java
public record Num(int value) implements Expr {

    @Override
    public <R> R accept(ExprVisitor<R> visitor) {
        return visitor.visitNumber(this); // l'élément choisit sa méthode visit
    }
}
```

Listing 4 – gof/Add.java — ConcreteElement

```java
public record Add(Expr left, Expr right) implements Expr {

    @Override
    public <R> R accept(ExprVisitor<R> visitor) {
        return visitor.visitAddition(this);
    }
}
```

Listing 5 – gof/Mul.java — ConcreteElement

```java
public record Mul(Expr left, Expr right) implements Expr {

    @Override
    public <R> R accept(ExprVisitor<R> visitor) {
        return visitor.visitMultiplication(this);
    }
}
```

Deux opérations sur la même structure, donc deux visiteurs concrets. Ajouter une troisième opération (compter les nœuds, simplifier…) consisterait à écrire une troisième classe, sans toucher à `Num`, `Add` ni `Mul`.

Listing 6 – gof/Evaluator.java — ConcreteVisitor n° 1

```java
public class Evaluator implements ExprVisitor<Integer> {

    @Override
```

```
    public Integer visitNumber(Num number) {
        return number.value();
    }

    @Override
    public Integer visitAddition(Add addition) {
        return addition.left().accept(this) + addition.right().accept(this);
    }

    @Override
    public Integer visitMultiplication(Mul multiplication) {
        return multiplication.left().accept(this)
                * multiplication.right().accept(this);
    }
}
```

Listing 7 – gof/Printer.java (extrait) — ConcreteVisitor n° 2 : même forme, autre opération

```
public class Printer implements ExprVisitor<String> {

    @Override
    public String visitNumber(Num number) {
        return Integer.toString(number.value());
    }

    @Override
    public String visitAddition(Add addition) {
        return "(" + addition.left().accept(this)
                + " + " + addition.right().accept(this) + ")";
    }
```

Listing 8 – gof/GofDemo.java (extrait) — Client

```
    public static void main(String[] args) {
        // La structure d'objets (ObjectStructure) : un arbre d'expressions.
        Expr expr = new Mul(new Num(2), new Add(new Num(3), new Num(4)));

        // Même structure, deux opérations : deux visiteurs concrets.
        System.out.println("Printer   : " + expr.accept(new Printer()));
        System.out.println("Evaluator : " + expr.accept(new Evaluator()));

        // Ordre réel des visites (diagramme de séquence de l'annexe 3).
        System.out.println("Trace de l'evaluation :");
        expr.accept(new TracingEvaluator());
    }
```

`TracingEvaluator` (dans le même paquetage) hérite d'`Evaluator` et affiche chaque visite avec sa valeur : c'est la trace de la fin de la sortie.

```
Printer   : 2 * (3 + 4)
Evaluator : 14
Trace de l'evaluation :
  visitMultiplication
    visitNumber(2)
    <- 2
    visitAddition
      visitNumber(3)
      <- 3
      visitNumber(4)
      <- 4
    <- 7
  <- 14
```

2.4. Le double aiguillage à l'exécution

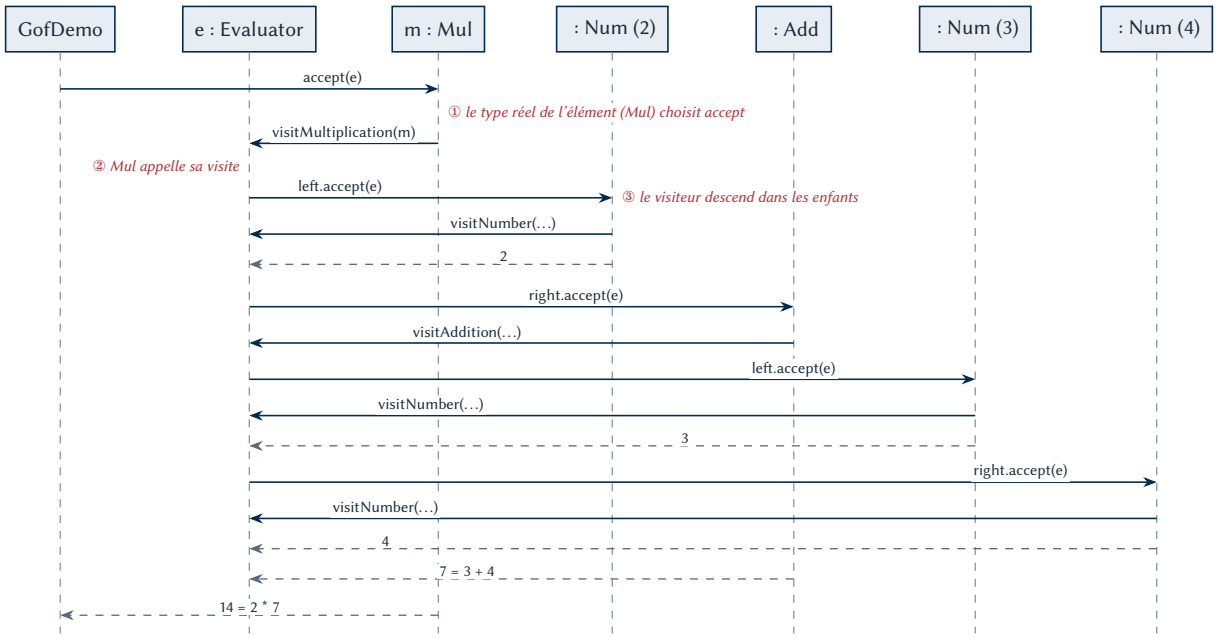


Figure 3 – Évaluation de `2 * (3 + 4)`. Retours simplifiés : le retour de `accept` et celui de la visite sont confondus. L'ordre des visites est celui de la trace ci-dessus.

1. **Le type réel de l'élément choisit `accept`.** `expr` est déclaré `Expr` ; à l'exécution, c'est `Mul.accept` qui s'exécute (liaison dynamique).
2. **`accept` dirige le visiteur vers l'opération de cet élément.** `Mul.accept` appelle `visitMultiplication(this)` ; l'objet visiteur réel (`Evaluator` ou `Printer`) décide de ce que fait cette méthode. L'opération exécutée dépend donc de **deux** types : c'est le *double dispatch*.
3. **Le parcours suit la structure, récursivement.** Ici, c'est le visiteur qui appelle `accept` sur `left` et `right`. JDT fait un autre choix (§ 3.2).

2.5. Forces et limites

FACILE	DIFFICILE
ajouter une **opération** : un nouveau visiteur, sans toucher aux nœuds	ajouter un **type de nœud** : une méthode de plus dans *tous* les visiteurs
regrouper une analyse dans une seule classe, avec son état	accéder à l'état des nœuds sans briser leur encapsulation

Pour un AST Java, le compromis est excellent : les types de nœuds sont fixés par la spécification du langage, les analyses se multiplient.

3. Le même patron dans JDT : `ASTVisitor`

3.1. Correspondance des rôles

RÔLE (GOF)	ARBRE D'EXPRESSIONS	JDT
Element	`Expr`	`ASTNode`
`accept`	`Expr.accept(ExprVisitor<R>)`	`ASTNode.accept(ASTVisitor)`
Visitor	`ExprVisitor<R>`	`ASTVisitor` : `visit(X)` et `endVisit(X)` pour chaque type de nœud X, avec des implémentations par défaut
ConcreteElement	`Num`, `Add`, `Mul`	`TypeDeclaration`, `MethodDeclaration`, `MethodInvocation`… (plus d'une centaine)
ConcreteVisitor	`Evaluator`, `Printer`	**vos** classes : `StructureVisitor`, `CallVisitor`…
Parcours des enfants	écrit dans le visiteur	fait par `ASTNode.accept`, si `visit` renvoie `true`

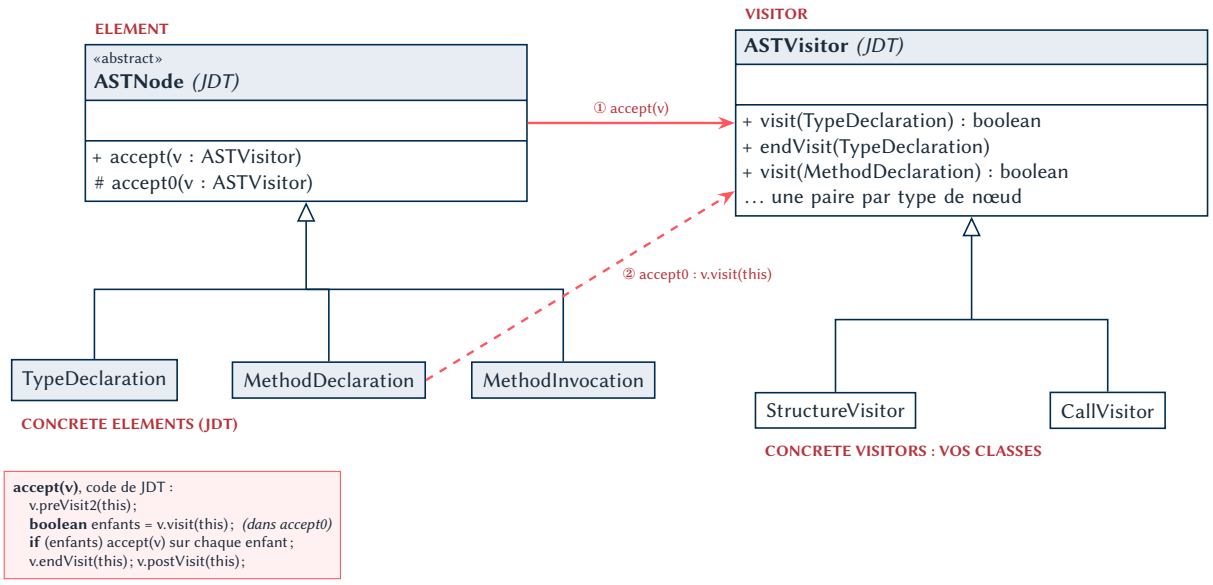


Figure 4 – Le patron dans JDT. Le parcours n'est plus écrit dans le visiteur : il est programmé une fois pour toutes dans `ASTNode.accept`.

3.2. La différence qui compte

QUI PARCOURT LES ENFANTS ?

- **Arbre d'expressions.** L'élément concret appelle explicitement la visite qui le concerne (`visitMultiplication(this)`), et **le visiteur** choisit de parcourir les enfants en appelant `left.accept(this)`.
- **JDT.** Chaque nœud appelle `v.visit(this)` : parmi les méthodes `visit` *surchargées*, le compilateur de JDT a choisi, d'après le type statique de `this`, celle du bon type de nœud. Puis **l'infrastructure de JDT** (`ASTNode.accept`) visite les enfants *si et seulement si* `visit` a renvoyé `true`, et appelle toujours `endVisit`. Votre visiteur ne parcourt rien lui-même : il *décide* seulement, nœud par nœud, s'il faut descendre.

3.3. Parcours et élagage : `return true` / `return false`

Le projet contient deux programmes sur le même petit fichier (une méthode `label` avec un `if`, une méthode `task` qui renvoie une classe anonyme contenant elle aussi un `if`) : `JdtVisitorDemo` compte les `if` et les chaînes par méthode en élaguant les classes anonymes, `JdtTraceDemo` affiche chaque `visit` avec la valeur renvoyée et chaque `endVisit`. Le visiteur de trace décide, pour chaque type de nœud, de descendre ou non (figure 5 et extraits ci-dessous) :

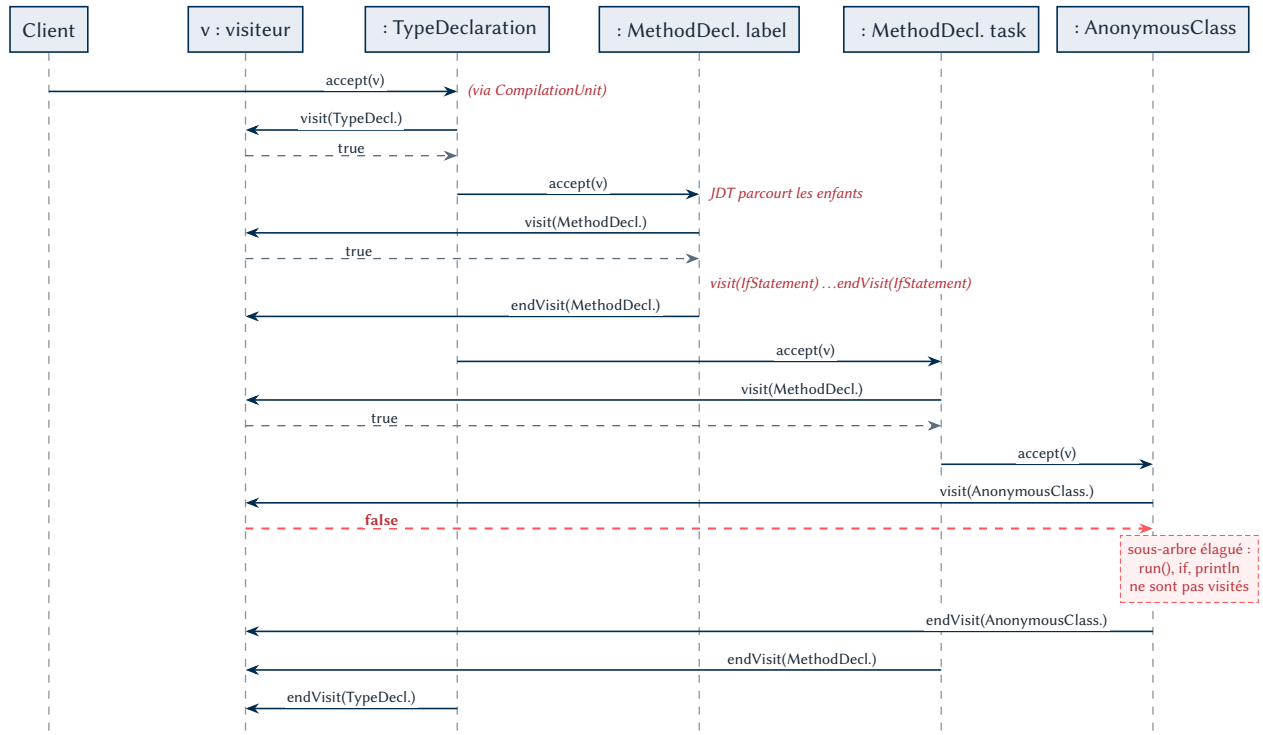


Figure 5 – Trace de `JdtTraceDemo` en diagramme de séquence. JDT descend dans les enfants tant que `visit` renvoie `true` ; `false` sur la classe anonyme élague son sous-arbre (`run()`, son `if`), mais `endVisit` est tout de même appelé.

Listing 9 – `jdt/JdtTraceDemo.java` (extrait) — la décision est la valeur renvoyée

```
36              private boolean enter(String node, boolean children) {
37                  System.out.println("  ".repeat(depth) + "visit(" + node
38                          + ") -> " + children);
39                  depth++;
40                  return children;
41              }
```

Listing 10 – `jdt/JdtTraceDemo.java` (extrait) — élagage de la classe anonyme

```
79              @Override
80              public boolean visit(AnonymousClassDeclaration node) {
81                  return enter("AnonymousClassDeclaration", false); // élagage
82              }
83
84              @Override
85              public void endVisit(AnonymousClassDeclaration node) {
86                  leave("AnonymousClassDeclaration");
87              }
```

```
visit(TypeDeclaration Shop) -> true
  visit(MethodDeclaration label) -> true
    visit(IfStatement) -> true
    endVisit(IfStatement)
  endVisit(MethodDeclaration label)
  visit(MethodDeclaration task) -> true
    visit(AnonymousClassDeclaration) -> false
    endVisit(AnonymousClassDeclaration)
  endVisit(MethodDeclaration task)
endVisit(TypeDeclaration Shop)
```

4. Utiliser un visiteur dans la chaîne du TP

Il y a plusieurs façons correctes de créer et d'employer un `ASTVisitor`. Aucune n'est imposée ; elles ne servent pas aux mêmes moments. Les extraits viennent du paquetage `variants`, appliqué par `VariantsDemo` à un petit programme de deux fichiers. Ses visiteurs sont **volontairement simplifiés** (pas d'énumérations, de types imbriqués ni de *bindings*) : ils illustrent l'organisation, pas les étapes A2 et A3.

	FORME	QUAND L'UTILISER	LIMITE
1	visiteur anonyme, ad hoc	A1, une expérience locale, un test	état dans des variables englobantes ; ne se réutilise pas
2	classe nommée dédiée	une préoccupation réutilisable, qui garde son état	résultats enfermés dans le visiteur
3	visiteur + accumulateur externe	écrire directement dans **votre** modèle de faits	un accumulateur à concevoir (annexe 1)
4	plusieurs visiteurs spécialisés	structure et appels séparés, testés séparément	plusieurs parcours de chaque unité (sans coût notable ici)
5	un seul visiteur combiné	un seul parcours	état partagé, élagage impossible, couplage ; **pas** automatiquement meilleur
6	orchestration de la chaîne	l'organisation recommandée pour B1	—

4.1. Forme 1 : visiteur anonyme

Listing 11 – `variants/VariantsDemo.java` (extrait)

```
35          // 1) Visiteur anonyme, ad hoc : une observation locale et jetable.
36          int[] calls = new int[1];
37          for (CompilationUnit unit : units) {
38              unit.accept(new ASTVisitor() {
39                  @Override
40                  public boolean visit(MethodInvocation node) {
41                      calls[0]++;
42                      return true;
43                  }
44              });
45          }
46          System.out.println("1) anonyme     : " + calls[0] + " appels");
```

4.2. Forme 2 : classe nommée qui garde son état

Listing 12 – variants/MethodNameCollector.java

```java
public class MethodNameCollector extends ASTVisitor {

    private final List<String> names = new ArrayList<>();

    @Override
    public boolean visit(MethodDeclaration node) {
        names.add(node.getName().getIdentifier());
        return true;
    }

    public List<String> names() {
        return names;
    }
}
```

4.3. Forme 3 : visiteur qui écrit dans un accumulateur externe

Le visiteur *observe* les nœuds JDT ; ce qu'il en tire est écrit dans un objet qui ne connaît pas JDT. Le corps des méthodes ne sert pas à la structure : `return false` l'élague.

Listing 13 – variants/StructureVisitor.java

```java

    private final ToyFacts.Builder facts;
    private String currentType = "?";

    public StructureVisitor(ToyFacts.Builder facts) {
        this.facts = facts;
    }

    @Override
    public boolean visit(TypeDeclaration node) {
        currentType = node.getName().getIdentifier();
        facts.addType(currentType);
        return true;
    }

    @Override
    public boolean visit(MethodDeclaration node) {
        facts.addMethod(currentType, node.getName().getIdentifier());
        return false; // le corps ne concerne pas la structure : élagué
    }
}
```

Listing 14 – variants/ToyFacts.java (extrait) — l'accumulateur ; addMethod, addCall et build suivent

```java
    public static final class Builder {
        private final List<String> types = new ArrayList<>();
        private final List<String> methods = new ArrayList<>();
        private final List<String> calls = new ArrayList<>();

        public void addType(String type) {
            types.add(type);
        }
```

4.4. Forme 4 : plusieurs visiteurs spécialisés

Listing 15 – `variants/CallVisitor.java` (extrait)

```java

    @Override
    public boolean visit(MethodDeclaration node) {
        currentMethod = node.getName().getIdentifier();
        return true;
    }

    @Override
    public boolean visit(MethodInvocation node) {
        facts.addCall(currentMethod, node.getName().getIdentifier());
        return true;
    }
```

4.5. Forme 5 : un seul visiteur combiné

Possible, et même un peu plus rapide. Mais la structure et les appels partagent maintenant `currentType` et `currentMethod`, et `visit(MethodDeclaration)` doit renvoyer `true` pour atteindre les appels :

Listing 16 – `variants/CombinedVisitor.java` (extrait)

```java
    @Override
    public boolean visit(MethodDeclaration node) {
        currentMethod = node.getName().getIdentifier();
        facts.addMethod(currentType, currentMethod);
        return true; // il faut descendre : les appels sont dans le corps
    }

    @Override
    public boolean visit(MethodInvocation node) {
        facts.addCall(currentMethod, node.getName().getIdentifier());
        return true;
    }
```

4.6. Forme 6 : l'orchestration, la chaîne recommandée

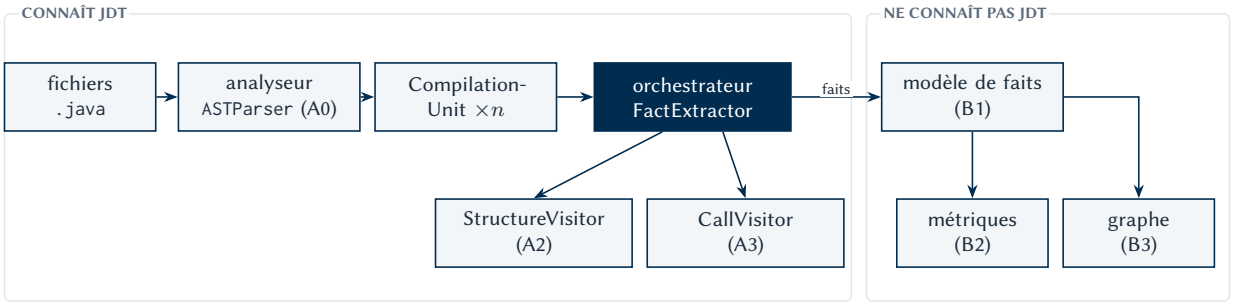


Figure 6 – La chaîne du TP : seuls l'analyseur, l'orchestrateur et les visiteurs connaissent JDT ; les traitements de B2 et B3 ne lisent que le modèle de faits.

Listing 17 – `variants/FactExtractor.java` (extrait)

```java
    public static ToyFacts extract(List<CompilationUnit> units) {
        ToyFacts.Builder facts = new ToyFacts.Builder();
        for (CompilationUnit unit : units) {
            unit.accept(new StructureVisitor(facts)); // passe 1
            unit.accept(new CallVisitor(facts));      // passe 2
        }
        return facts.build();
    }
```

Listing 18 – `variants/ToyMetrics.java` (extrait) — un traitement sans JDT

```java
    public static Map<String, Long> methodsPerType(ToyFacts facts) {
        return facts.methods().stream().collect(Collectors.groupingBy(
                id -> id.substring(0, id.indexOf('#')),
                TreeMap::new, Collectors.counting()));
    }
```

4.7. Résultat de `VariantsDemo`

```
1) anonyme     : 4 appels
2) nommee      : [add, log, main]
3-4-6) chaine  : types   [Library, App]
                 methods [Library#add, Library#log, App#main]
                 calls   [add -> add, add -> log, log -> println, main -> add]
5) combine     : meme resultat = true
   traitement  : {App=1, Library=2}
```

Les formes 3, 4 et 6 et la forme 5 produisent exactement les mêmes faits ; le choix relève de la lisibilité, du test et de l'évolution, et se justifie dans le compte-rendu.

5. Le visiteur à chaque étape du TP

ÉTAPE	RÔLE DU VISITEUR	FORME TYPIQUE
A1	observer l'AST : types de nœuds, profondeur, effet de `return false`	visiteur anonyme (forme 1) ou petite classe nommée
A2	structure : types, attributs, méthodes, hiérarchie	`StructureVisitor` nommé (formes 2 ou 3)
A3	appels, *bindings*, type statique du receveur	`CallVisitor` spécialisé (forme 4)
B1	les visiteurs remplissent **votre** modèle de faits	accumulateur externe et orchestrateur (formes 3 et 6)
B2 B3	**aucun** `ASTVisitor` : on lit le modèle de faits	traitements sans import `org.eclipse.jdt`

6. Dans Spoon : scanner, filtre, processeur

JDT OBLIGATOIRE, SPOON EN BONUS

Le TP se fait avec `ASTVisitor` de JDT. Spoon (partie C) propose des formes de parcours voisines, de plus haut niveau.

Chaque `CtElement` de Spoon a une méthode `accept(CtVisitor)`, et `CtScanner` est un visiteur qui parcourt tout le sous-arbre. Spoon ajoute deux formes plus déclaratives.

Listing 19 – spoondemo/SpoonDemo.java (extrait : le scanner ; filtre et processeur suivent dans le fichier)

```
37          // 1) Scanner : le visiteur de Spoon. On redéfinit visitCtXxx et on
38          //    rappelle super pour descendre (le « return true » de JDT).
39          int[] ifs = new int[1];
40          shop.accept(new CtScanner() {
41              @Override
42              public void visitCtIf(CtIf ifElement) {
43                  ifs[0]++;
44                  super.visitCtIf(ifElement);
45              }
46          });
47          System.out.println("1) scanner    : " + ifs[0] + " if");
```

```
1) scanner    : 1 if
2) filtre     : methode label lignes 2-7
2) filtre     : methode twice lignes 8-8
3) processeur : catch vide ligne 5
```

FORME	PRINCIPE	ÉQUIVALENT JDT
`CtScanner`	redéfinir `visitCtXxx`, rappeler `super` pour descendre	`visit` qui renvoie `true`
filtre, requête	`getElements(new TypeFilter<>(CtMethod.class))`, `filterChildren(...).map(...).list()`	un visiteur qui accumule des nœuds dans une liste
processeur	`AbstractProcessor<E>` : `isToBeProcessed`, puis `process` pour chaque `E`	visiteur spécialisé, appliqué par un lanceur

7. Plus d'exemples sur les design patterns ?

> **DÉPÔT GITHUB POUR LES DESIGN PATTERNS**
> Plusieurs design patterns sont mises en oeuvre à travers un exemple et une base de code similairement au patron Visiteur ci-dessus. Pour les comprendre et aller plus loin, consulter le dépôt GitHub anonbnr/org.anonbnr.design_patterns.

8. Références

- E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns : Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 — chapitre « Visitor ».
- Refactoring.Guru — Visitor (présentation illustrée, avec double dispatch).
- Javadoc d'`ASTVisitor` (ordre d'appel, élagage).
- Spoon — processeurs et filtres.